Il linguaggio SQL con MySQL

DDL · DML · DCL · transazioni · stored routine · trigger

Il percorso

1. SQL e MySQL: caratteristiche generali e regole di scrittura
2. **Tipi di dato** in MySQL
3. **DDL**: creare e modificare database, tabelle, vincoli, indici
4. **DML**: inserire, modificare, cancellare
5. **SELECT**: interrogazioni, JOIN, raggruppamenti, subquery, viste
6. **DCL**: utenti, privilegi, ruoli
7. **Transazioni**, concorrenza e **record fantasma**
8. **Stored procedure**, **funzioni** e **trigger**
9. Un caso completo in stile esame di maturità

1 · SQL e MySQL

Che cos'è il linguaggio, come lo esegue MySQL, come si scrive

SQL: un linguaggio dichiarativo

> **SQL** (*Structured Query Language*) è il linguaggio standard per **definire**, **manipolare**, **interrogare** e **proteggere** i dati di un database relazionale.

- **Dichiarativo** (non procedurale): si descrive **che cosa** si vuole ottenere, non **come** trovarlo. Il percorso di accesso ai dati lo sceglie l'**ottimizzatore** del DBMS
- Nato in IBM negli anni '70 (System R) → standard **ANSI 1986** / **ISO 1987** → SQL2 (1992), SQL3 (1999) … fino a **SQL:2023**
- Ogni DBMS (MySQL, PostgreSQL, Oracle, SQL Server…) rispetta il **nucleo standard** e aggiunge **estensioni** proprie

In queste slide: **tutto in sintassi MySQL 8 / 9**, motore **InnoDB**.

Le categorie di comandi

Categoria	Scopo	Comandi MySQL principali
DDL – *Data Definition Language*	definire la struttura	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`
DML – *Data Manipulation Language*	gestire i dati	`INSERT`, `UPDATE`, `DELETE`
DQL – *Data Query Language*	interrogare	`SELECT`
DCL – *Data Control Language*	sicurezza e accessi	`CREATE USER`, `GRANT`, `REVOKE`, `CREATE ROLE`
TCL – *Transaction Control Language*	gestire le transazioni	`START TRANSACTION`, `COMMIT`, `ROLLBACK`, `SAVEPOINT`

Molti testi includono `SELECT` nel DML: è solo una questione di classificazione.

L'ambiente MySQL

- **Server** `mysqld`: gestisce i database e risponde alle richieste
- **Client**: riga di comando `mysql -u root -p`, **MySQL Workbench**, **phpMyAdmin**, oppure un'applicazione (PHP, Python, Java...)
- **Storage engine**: il "motore" che salva fisicamente le tabelle. Quello predefinito è **InnoDB**, che supporta **chiavi esterne**, **transazioni** e **lock a livello di riga**

```
SELECT VERSION();          -- versione del server
SHOW DATABASES;            -- elenco dei database
USE azienda;               -- seleziona il database di lavoro
SHOW TABLES;               -- tabelle del database corrente
SHOW ENGINES;              -- motori disponibili (InnoDB è DEFAULT)
```

Nota: alcuni pacchetti per la didattica (es. XAMPP) installano **MariaDB**, un "fork" di MySQL: la sintassi vista qui è in larghissima parte compatibile.

Regole di scrittura in MySQL

- Ogni istruzione termina con `;` e può occupare più righe (formato libero)
- Le **parole chiave** non distinguono maiuscole e minuscole (`SELECT` = `select`)
- **Identificatori** (nomi di database, tabelle, colonne): fino a **64 caratteri**. Se contengono spazi o parole riservate vanno tra **backtick**: `` `Data ordine` ``
- I nomi delle **tabelle** sono *case-sensitive* su Linux e non su Windows → usare **sempre lo stesso stile**
- **Stringhe** e **date** tra apici singoli: `'Torino'`, `'2026-11-10'`
- **Dot notation**: `Tabella.Colonna`, oppure `database.Tabella.Colonna`

```
-- commento fino a fine riga (serve lo spazio dopo --)
#  commento fino a fine riga (stile MySQL)
/* commento
   su più righe */
```

Il valore NULL

> **NULL** indica un valore **sconosciuto o mancante**. Non è zero e non è la stringa vuota `''`.

- Qualsiasi confronto con NULL dà **UNKNOWN** (logica a tre valori): `NULL = NULL` **non** è vero
- Per controllarlo si usano solo `IS NULL` e `IS NOT NULL`
- Nelle espressioni NULL "contagia" il risultato: `Stipendio + NULL` → NULL
- In MySQL, negli ordinamenti NULL compare **all'inizio** in ordine crescente e **alla fine** in ordine decrescente

```
SELECT NULL = NULL, NULL IS NULL, 10 + NULL, IFNULL(NULL, 0);
-- risultato:  NULL      1          NULL        0
```

2 · I tipi di dato in MySQL

Scegliere il tipo giusto = meno spazio, più controlli, meno errori

Tipi numerici interi

Tipo	Byte	Intervallo con segno	Intervallo `UNSIGNED`
`TINYINT`	1	−128 … 127	0 … 255
`SMALLINT`	2	−32.768 … 32.767	0 … 65.535
`MEDIUMINT`	3	−8.388.608 … 8.388.607	0 … 16.777.215
`INT` (`INTEGER`)	4	−2.147.483.648 … 2.147.483.647	0 … 4.294.967.295
`BIGINT`	8	≈ ±9,2 × 10^{18}	0 … ≈ 1,8 × 10^{19}

- **`UNSIGNED`** esclude i negativi e raddoppia il massimo: ideale per codici ed età
- **`AUTO_INCREMENT`**: il valore viene generato automaticamente (1, 2, 3…), tipico delle chiavi primarie surrogate
- Scrivere `INT(11)` è **deprecato**: il numero tra parentesi non limita i valori

Tipi numerici decimali

Tipo	Caratteristiche	Uso tipico
`DECIMAL(p,s)` (`NUMERIC`)	**valore esatto**: *p* cifre totali (max 65), *s* dopo la virgola	**importi in euro**, percentuali, voti
`FLOAT`	virgola mobile, 4 byte, ≈ 7 cifre significative	misure scientifiche approssimate
`DOUBLE`	virgola mobile, 8 byte, ≈ 15 cifre significative	calcoli scientifici

`DECIMAL(9,2)` → da −9.999.999,99 a 9.999.999,99 (7 cifre intere + 2 decimali)

> I numeri in virgola mobile sono **approssimati**: un importo in `DOUBLE` può dare risultati come `0.30000000000000004`. **Per i soldi si usa sempre `DECIMAL`.**

Nelle costanti il separatore decimale è il **punto**: `1250.50`, `-1.2E+7`.

Tipi stringa

Tipo	Caratteristiche	Esempio d'uso
`CHAR(n)`	lunghezza **fissa**, n ≤ 255	codice fiscale `CHAR(16)`, CAP `CHAR(5)`, sigla provincia `CHAR(2)`
`VARCHAR(n)`	lunghezza **variabile** fino a n caratteri	nome, cognome, email, indirizzo
`TEXT`	testi lunghi (fino a 64 KB)	descrizioni, note, recensioni
`ENUM('a','b',…)`	un solo valore da un elenco fisso	`ENUM('M','F','X')`, stato di un ordine

- Il set di caratteri predefinito di MySQL 8 è **`utf8mb4`**: gestisce accenti ed emoji
- Con la *collation* predefinita i confronti **ignorano maiuscole e accenti**: `'rossi' = 'ROSSI'` è vero
- Numeri che non servono nei calcoli (telefono, CAP, partita IVA) → **stringhe**, così gli zeri iniziali non si perdono

Tipi data e ora

Tipo	Formato	Intervallo / note
`DATE`	`'AAAA-MM-GG'`	dal `1000-01-01` al `9999-12-31`
`TIME`	`'hh:mm:ss'`	anche durate (es. `'25:30:00'`)
`DATETIME`	`'AAAA-MM-GG hh:mm:ss'`	data e ora "così come sono"
`TIMESTAMP`	come `DATETIME`	salvato in UTC; dal 1970 al 2038; utile per "ultima modifica"
`YEAR`	`AAAA`	dal 1901 al 2155

```sql
SELECT CURDATE(), NOW(), YEAR('2026-11-10'),
       DATEDIFF('2026-12-25', CURDATE()) AS GiorniANatale,
       TIMESTAMPDIFF(YEAR, '2008-03-15', CURDATE()) AS Eta;
```

Attenzione: in MySQL le date si scrivono **anno-mese-giorno**, non `10/11/2026`.

Booleani e scelta del tipo giusto

`BOOLEAN` (o `BOOL`) in MySQL è un sinonimo di **`TINYINT(1)`**: `TRUE` = 1, `FALSE` = 0.

Dato	Tipo consigliato	Perché
Matricola, codice cliente	`INT UNSIGNED AUTO_INCREMENT`	chiave surrogata generata dal DBMS
Codice fiscale	`CHAR(16)`	lunghezza fissa
Email	`VARCHAR(100)`	lunghezza variabile
Telefono, CAP	`VARCHAR(15)`, `CHAR(5)`	non si fanno calcoli; zeri iniziali
Prezzo, stipendio	`DECIMAL(8,2)` / `DECIMAL(9,2)`	valore esatto
Voto (es. 7.5)	`DECIMAL(3,1)`	un decimale esatto
Data di nascita	`DATE`	permette calcoli su date
Pagato sì/no	`BOOLEAN`	due soli valori

3 · DDL: definire la struttura

`CREATE` · `ALTER` · `DROP` · vincoli · chiavi esterne · indici

L'esempio guida: un'azienda

Un'azienda è divisa in **dipartimenti**; ogni **impiegato** appartiene a un dipartimento e lavora su uno o più **progetti** con un **ruolo** e una **percentuale** del proprio tempo.

- Dipartimento **1:N** Impiegato (un impiegato può non essere ancora assegnato)
- Impiegato **N:N** Progetto (associazione con attributi Ruolo e Percentuale)

Schema logico (chiave primaria, *chiave esterna*):

- Dipartimenti (Codice, Descrizione, Sede)
- Impiegati (ID, Nome, Cognome, Residenza, Stipendio, *Dipartimento*)
- Progetti (CodProg, NomeProgetto, Durata, Budget)
- Partecipazioni (*ID*, *CodProg*, Ruolo, Percentuale)

Creare il database

```
CREATE DATABASE IF NOT EXISTS azienda
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_0900_ai_ci;   -- ai = accent insensitive, ci = case insensitive

USE azienda;

-- per eliminarlo (con TUTTE le tabelle e i dati!)
DROP DATABASE IF EXISTS azienda;
```

- `IF NOT EXISTS` / `IF EXISTS` evitano errori se l'oggetto esiste già (o non esiste)
- Lo **script di creazione** si scrive in un file `.sql` e si esegue tutto insieme: ordine consigliato → `DROP` → `CREATE DATABASE` → tabelle **padre** → tabelle **figlie**

CREATE TABLE: la tabella Dipartimenti

```
CREATE TABLE Dipartimenti (
  Codice       CHAR(5)      NOT NULL,
  Descrizione  VARCHAR(40)  NOT NULL,
  Sede         VARCHAR(30),
  PRIMARY KEY (Codice),
  UNIQUE (Descrizione)
) ENGINE = InnoDB;
```

- `PRIMARY KEY`: identifica in modo univoco la riga; implica **NOT NULL** e **nessun duplicato**
- `UNIQUE`: niente duplicati (Descrizione è una **chiave alternativa**)
- `NOT NULL`: campo **obbligatorio**
- `ENGINE = InnoDB`: è già il default, scriverlo rende lo script esplicito

CREATE TABLE: la tabella Impiegati

```sql
CREATE TABLE Impiegati (
  ID            SMALLINT UNSIGNED  AUTO_INCREMENT,
  Nome          VARCHAR(20)   NOT NULL,
  Cognome       VARCHAR(30)   NOT NULL,
  Residenza     VARCHAR(30)   DEFAULT 'Non indicata',
  Stipendio     DECIMAL(9,2)  CHECK (Stipendio > 0),
  Dipartimento  CHAR(5),                      -- NULL = non ancora assegnato
  PRIMARY KEY (ID),
  UNIQUE (Cognome, Nome, Dipartimento),
  CONSTRAINT fk_imp_dip
    FOREIGN KEY (Dipartimento) REFERENCES Dipartimenti (Codice)
    ON DELETE SET NULL
    ON UPDATE CASCADE
) ENGINE = InnoDB;
```

- `AUTO_INCREMENT`: MySQL assegna l'ID; `DEFAULT`: valore usato se non se ne fornisce uno
- `CHECK`: condizione che ogni riga deve rispettare (applicata davvero da **MySQL 8.0.16**)
- `CONSTRAINT nome`: dare un nome al vincolo permette di eliminarlo o modificarlo in seguito

Riepilogo dei vincoli

Vincolo	Significato	Errore MySQL se violato
`PRIMARY KEY`	univoco e non nullo, uno per tabella	`1062 Duplicate entry`
`UNIQUE`	nessun duplicato (NULL ammessi)	`1062 Duplicate entry`
`NOT NULL`	valore obbligatorio	`1048 Column cannot be null`
`DEFAULT valore`	valore se non specificato	–
`CHECK (condizione)`	regola sui valori della riga	`3819 Check constraint is violated`
`AUTO_INCREMENT`	numerazione automatica	–
`FOREIGN KEY … REFERENCES`	**integrità referenziale**	`1452` (figlio senza padre) / `1451` (padre con figli)

Un vincolo su **una** colonna si può scrivere accanto alla colonna; su **più** colonne va scritto in una riga a parte: `PRIMARY KEY (ID, CodProg)`.

Chiavi esterne in MySQL: attenzione!

In **MySQL 8.x** una chiave esterna scritta *accanto alla colonna*

```
Dipartimento CHAR(5) REFERENCES Dipartimenti(Codice)
```

viene **accettata ma ignorata**: nessun errore, nessun vincolo! Viene applicata solo da MySQL 9.0.

Usa **sempre** la forma separata, valida in tutte le versioni:

```
FOREIGN KEY (Dipartimento) REFERENCES Dipartimenti (Codice)
```

Altre regole:

- solo con **InnoDB**
- colonna figlia e colonna padre con **tipo compatibile** (stesso tipo, stessa lunghezza, stesso `UNSIGNED`)
- la tabella **padre** va creata **prima** della figlia
- MySQL crea in automatico un **indice** sulla colonna della chiave esterna

Le azioni referenziali

Che cosa succede ai figli quando cambia o sparisce il padre?

Clausola	Effetto su `Impiegati` quando cancello/modifico un dipartimento
`RESTRICT` / `NO ACTION` (**default**)	l'operazione è **vietata** se ci sono impiegati collegati
`CASCADE`	`ON DELETE`: vengono **cancellati** anche gli impiegati · `ON UPDATE`: il nuovo codice si **propaga**
`SET NULL`	il campo `Dipartimento` degli impiegati diventa **NULL** (la colonna deve ammettere NULL)
`SET DEFAULT`	**non supportato** da InnoDB

Esempio con `ON DELETE SET NULL ON UPDATE CASCADE`:

- `DELETE FROM Dipartimenti WHERE Codice = 'Direz';` → gli impiegati della Direzione restano, ma senza dipartimento
- `UPDATE Dipartimenti SET Codice = 'DG' WHERE Codice = 'Direz';` → i loro record passano automaticamente a `'DG'`

Progetti e Partecipazioni (chiave composta)

```sql
CREATE TABLE Progetti (
  CodProg       CHAR(10)       PRIMARY KEY,
  NomeProgetto  VARCHAR(60)    NOT NULL,
  Durata        TINYINT UNSIGNED,              -- in mesi
  Budget        DECIMAL(12,2)  CHECK (Budget >= 0)
);

CREATE TABLE Partecipazioni (
  ID           SMALLINT UNSIGNED,
  CodProg      CHAR(10),
  Ruolo        VARCHAR(30)  NOT NULL,
  Percentuale  TINYINT UNSIGNED  CHECK (Percentuale BETWEEN 1 AND 100),
  PRIMARY KEY (ID, CodProg),
  FOREIGN KEY (ID)      REFERENCES Impiegati (ID) ON DELETE CASCADE,
  FOREIGN KEY (CodProg) REFERENCES Progetti (CodProg)
);
```

`ID` è `SMALLINT UNSIGNED` come in `Impiegati`: tipi **identici**, altrimenti errore `3780`.

ALTER TABLE: modificare la struttura

```
ALTER TABLE Impiegati ADD COLUMN Nascita DATE AFTER Cognome;    -- aggiunge una colonna
ALTER TABLE Impiegati DROP COLUMN Residenza;                     -- elimina colonna e dati
ALTER TABLE Impiegati MODIFY Nome VARCHAR(40) NOT NULL;          -- cambia tipo/vincoli
ALTER TABLE Impiegati RENAME COLUMN Nascita TO DataNascita;      -- rinomina
ALTER TABLE Progetti  ADD CONSTRAINT uq_nome UNIQUE (NomeProgetto);
ALTER TABLE Impiegati DROP FOREIGN KEY fk_imp_dip;               -- serve il nome del vincolo
ALTER TABLE Impiegati
  ADD CONSTRAINT fk_imp_dip FOREIGN KEY (Dipartimento)
      REFERENCES Dipartimenti (Codice) ON DELETE RESTRICT;
RENAME TABLE Progetti TO Commesse;                               -- rinomina la tabella
```

Con `MODIFY`, se i dati esistenti non rispettano il nuovo tipo (es. stringhe troppo lunghe), MySQL in modalità *strict* **blocca** l'operazione.

Indici

> Un **indice** è una struttura (B-tree) che velocizza la **ricerca** e l'**ordinamento** sui valori di una o più colonne, a costo di spazio e di inserimenti un po' più lenti.

```sql
CREATE INDEX idx_cognome ON Impiegati (Cognome, Nome);
CREATE UNIQUE INDEX idx_email ON Clienti (Email);   -- indice + vincolo di unicità
SHOW INDEX FROM Impiegati;
DROP INDEX idx_cognome ON Impiegati;
```

- MySQL crea **da solo** gli indici per `PRIMARY KEY`, `UNIQUE` e `FOREIGN KEY`
- Conviene indicizzare le colonne usate spesso in `WHERE`, `JOIN` e `ORDER BY`
- Non conviene indicizzare colonne con pochi valori diversi (es. `Sesso`)

Ispezionare ed eliminare

```
DESCRIBE Impiegati;                -- colonne, tipi, chiavi, default
SHOW CREATE TABLE Impiegati;       -- lo script completo, con i nomi dei vincoli
SHOW TABLES;

DROP TABLE IF EXISTS Partecipazioni;   -- struttura + dati
TRUNCATE TABLE Partecipazioni;          -- svuota: struttura resta, AUTO_INCREMENT riparte da 1
```

L'ordine conta: non si può eliminare `Dipartimenti` finché esiste `Impiegati` che la referenzia (errore `3730`). Si eliminano prima le **figlie**, poi i **padri**.

4 · DML: manipolare i dati

`INSERT` · `UPDATE` · `DELETE`

INSERT

```sql
-- una riga, indicando le colonne (forma consigliata)
INSERT INTO Dipartimenti (Codice, Descrizione, Sede)
VALUES ('Prod', 'Produzione', 'Torino');

-- più righe in un solo comando
INSERT INTO Dipartimenti (Codice, Descrizione, Sede) VALUES
  ('Mag',  'Magazzino',          'Torino'),
  ('R&S',  'Ricerca e Sviluppo', 'Milano'),
  ('Direz','Direzione Generale', 'Roma');

-- ID omesso: lo genera AUTO_INCREMENT; Residenza omessa: prende il DEFAULT
INSERT INTO Impiegati (Nome, Cognome, Stipendio, Dipartimento)
VALUES ('Mario', 'Rossini', 31500.00, 'Mag');

SELECT LAST_INSERT_ID();   -- l'ID appena generato
```

INSERT … SELECT e CREATE TABLE … AS SELECT

Copiare dati da una query, invece di scriverli a mano:

```sql
-- accoda a una tabella esistente le righe estratte da un'altra
INSERT INTO Fornitori (CodForn, NomeSocieta, Citta, Nazione)
SELECT CodForn, NomeSocieta, Citta, Nazione
FROM NuoviFornitori
WHERE Contatto = 'Rossi';

-- crea una NUOVA tabella con struttura e dati presi dalla query
CREATE TABLE FornitoriUSA AS
SELECT * FROM Fornitori
WHERE Nazione = 'USA';
```

Con `CREATE TABLE … AS SELECT` le colonne e i tipi vengono copiati, ma **non** le chiavi primarie e le chiavi esterne: vanno aggiunte con `ALTER TABLE`.

UPDATE

```sql
-- assegna l'impiegato 20 al dipartimento Produzione
UPDATE Impiegati
SET Dipartimento = 'Prod'
WHERE ID = 20;

-- più colonne in un comando
UPDATE Impiegati
SET Residenza = 'Torino', Stipendio = 33000
WHERE ID = 20;

-- su un GRUPPO di righe: +5% a tutto il dipartimento Produzione
UPDATE Impiegati
SET Stipendio = Stipendio * 1.05
WHERE Dipartimento = 'Prod';
```

Senza `WHERE`, l'`UPDATE` modifica **tutte** le righe. MySQL Workbench attiva per sicurezza il *safe update mode* (errore `1175`), che blocca gli UPDATE/DELETE senza una condizione sulla chiave.

DELETE

```sql
DELETE FROM Impiegati WHERE ID = 20;                -- una riga
DELETE FROM Impiegati WHERE Dipartimento = 'R&S';   -- un gruppo di righe
DELETE FROM Impiegati;                              -- TUTTE le righe (struttura resta)
```

	`DELETE FROM t`	`TRUNCATE TABLE t`
Tipo	DML	DDL
`WHERE`	sì	no (svuota tutto)
Annullabile con `ROLLBACK`	sì	no
`AUTO_INCREMENT`	continua	riparte da 1
Attiva i trigger `DELETE`	sì	no

Se un dipartimento ha impiegati e la FK è `RESTRICT`, il `DELETE` fallisce con l'errore **`1451`**.

5 · Le interrogazioni con SELECT

Dalla singola tabella ai JOIN, raggruppamenti, subquery e viste

La struttura di SELECT

```
SELECT   [DISTINCT] colonne / espressioni     -- 5. proiezione
FROM     tabelle e JOIN                       -- 1. da dove
WHERE    condizione sulle righe               -- 2. selezione
GROUP BY colonne di raggruppamento            -- 3. raggruppamento
HAVING   condizione sui gruppi                -- 4. filtro sui gruppi
ORDER BY colonne [ASC | DESC]                 -- 6. ordinamento
LIMIT    n [OFFSET m];                        -- 7. quante righe
```

- Le clausole si **scrivono** in quest'ordine, ma il DBMS le **valuta** nell'ordine dei numeri a destra
- Per questo un alias definito nella `SELECT` **non** si può usare nella `WHERE` (si può invece in `ORDER BY`)
- Il risultato è sempre una **tabella** (virtuale)

Prime interrogazioni

```sql
-- proiezione + selezione: dipendenti di Produzione
SELECT Cognome, Nome, Residenza
FROM Impiegati
WHERE Dipartimento = 'Prod';

-- più condizioni con AND / OR / NOT
SELECT ID, Cognome, Nome
FROM Impiegati
WHERE Dipartimento = 'Prod' AND Residenza = 'Torino';

-- tutte le colonne
SELECT * FROM Impiegati WHERE Residenza = 'Roma';
```

Algebra relazionale ↔ SQL:

proiezione π → elenco dopo `SELECT` · **selezione σ** → `WHERE` · **congiunzione ⋈** → `JOIN` in `FROM`

DISTINCT, alias ed espressioni

```sql
-- località di residenza senza ripetizioni
SELECT DISTINCT Residenza FROM Impiegati;

-- rinominare una colonna e calcolare un'espressione
SELECT ID AS Matricola,
       CONCAT(Cognome, ' ', Nome)  AS Dipendente,
       Stipendio                   AS Attuale,
       ROUND(Stipendio * 1.05, 2)  AS ConAumento
FROM Impiegati
WHERE Dipartimento = 'Mag';

-- SELECT senza tabella: MySQL calcola e basta
SELECT 4 * 5 AS Area, SQRT(POW(3,2) + POW(4,2)) AS Ipotenusa;
```

In MySQL le stringhe si concatenano con **`CONCAT()`**: l'operatore `||` significa **OR logico**, non concatenazione.

ORDER BY e LIMIT

```sql
-- elenco alfabetico
SELECT Cognome, Nome, Stipendio
FROM Impiegati
ORDER BY Cognome, Nome;              -- ASC è il default

-- i 3 stipendi più alti
SELECT Cognome, Nome, Stipendio
FROM Impiegati
ORDER BY Stipendio DESC
LIMIT 3;

-- paginazione: 10 righe per pagina, terza pagina
SELECT ID, Cognome FROM Impiegati
ORDER BY ID
LIMIT 10 OFFSET 20;
```

`LIMIT` è un'estensione MySQL molto usata nelle applicazioni web (es. "ultimi 5 ordini").

Valori nulli nelle query

```sql
-- impiegati non ancora assegnati a un dipartimento
SELECT ID AS Matricola, Cognome, Nome
FROM Impiegati
WHERE Dipartimento IS NULL;
```

`WHERE Dipartimento = NULL` e `WHERE Dipartimento = ''` restituiscono **zero righe**: il confronto con NULL non è mai vero.

Sostituire NULL con un valore leggibile:

```sql
SELECT Cognome,
       IFNULL(Dipartimento, 'Da assegnare')  AS Reparto,
       COALESCE(Residenza, Sede, '?')         AS Luogo   -- primo valore non NULL
FROM Impiegati LEFT JOIN Dipartimenti ON Dipartimento = Codice;
```

Query parametriche in MySQL

Nella realtà il valore da cercare non è fisso: lo sceglie l'utente.

```
-- 1) variabili utente
SET @minimo = 55000;
SELECT Cognome, Nome, Stipendio FROM Impiegati WHERE Stipendio >= @minimo;

-- 2) prepared statement: il ? è il parametro
PREPARE q FROM 'SELECT Cognome, Nome FROM Impiegati WHERE Stipendio >= ?';
EXECUTE q USING @minimo;
DEALLOCATE PREPARE q;
```

Nella seconda prova la parte web (es. PHP) usa la stessa idea: `$pdo->prepare("… WHERE Stipendio >= ?")` ed `execute([$min])`. I parametri separati dal codice SQL proteggono dalla **SQL injection**.

Funzioni utili in MySQL

Categoria	Funzioni	Esempio → risultato
Stringhe	`UPPER`, `LOWER`, `LENGTH`, `SUBSTRING`, `TRIM`, `CONCAT`	`SUBSTRING('Torino',1,3)` → `'Tor'`
Numeri	`ROUND`, `TRUNCATE`, `ABS`, `MOD`, `CEIL`, `FLOOR`	`ROUND(12.456, 1)` → `12.5`
Date	`CURDATE`, `NOW`, `YEAR`, `MONTH`, `DATEDIFF`, `DATE_ADD`	`DATE_ADD('2026-11-10', INTERVAL 30 DAY)` → `2026-12-10`
Formattazione	`DATE_FORMAT`	`DATE_FORMAT(Data,'%d/%m/%Y')` → `10/11/2026`
Condizionali	`IF`, `IFNULL`, `CASE`	vedi sotto

```sql
SELECT Cognome, Stipendio,
       CASE WHEN Stipendio >= 50000 THEN 'Alto'
            WHEN Stipendio >= 30000 THEN 'Medio'
            ELSE 'Base' END AS Fascia
FROM Impiegati;
```

La congiunzione: INNER JOIN

Elenco dei dipendenti con i dati del loro dipartimento.

```sql
-- forma consigliata (SQL-92): condizione di join nella FROM
SELECT I.Cognome, I.Nome, D.Descrizione, D.Sede
FROM Impiegati AS I
  INNER JOIN Dipartimenti AS D ON I.Dipartimento = D.Codice;

-- forma "vecchia": tabelle separate da virgola, join nella WHERE
SELECT I.Cognome, I.Nome, D.Descrizione, D.Sede
FROM Impiegati I, Dipartimenti D
WHERE I.Dipartimento = D.Codice;
```

- Gli **alias** (`I`, `D`) accorciano il codice; definito l'alias, il nome originale non si usa più in quella query
- La forma con `JOIN … ON` separa la condizione di **congiunzione** da quelle di **selezione** in `WHERE`: più leggibile
- `JOIN` da solo equivale a `INNER JOIN`

JOIN + selezione e JOIN su più tabelle

```sql
-- dipendenti che lavorano in un dipartimento con sede a Roma
SELECT I.Cognome, I.Nome, D.Descrizione
FROM Impiegati I
  JOIN Dipartimenti D ON I.Dipartimento = D.Codice
WHERE D.Sede = 'Roma';

-- direttori di progetto, con nome del progetto (3 tabelle)
SELECT I.ID, I.Cognome, P.NomeProgetto, Pa.Percentuale
FROM Impiegati I
  JOIN Partecipazioni Pa ON I.ID = Pa.ID
  JOIN Progetti P        ON Pa.CodProg = P.CodProg
WHERE Pa.Ruolo = 'Direttore';

-- USING: se le colonne hanno lo stesso nome nelle due tabelle
SELECT ID, Cognome, CodProg FROM Impiegati JOIN Partecipazioni USING (ID);
```

Con `USING (ID)` nel risultato compare **una sola** colonna `ID`.

Join esterni: LEFT e RIGHT JOIN

L'`INNER JOIN` **perde** le righe senza corrispondenza: l'impiegato senza dipartimento e il dipartimento senza impiegati spariscono.

```sql
-- tutti gli impiegati, anche quelli senza dipartimento
SELECT I.Cognome, D.Descrizione
FROM Impiegati I LEFT JOIN Dipartimenti D ON I.Dipartimento = D.Codice;

-- problema dell'assenza: dipartimenti SENZA impiegati
SELECT D.Codice, D.Descrizione
FROM Dipartimenti D LEFT JOIN Impiegati I ON D.Codice = I.Dipartimento
WHERE I.ID IS NULL;
```

- `LEFT JOIN`: tutte le righe della tabella **a sinistra**, con NULL dove manca la corrispondenza
- `RIGHT JOIN`: lo stesso per la tabella **a destra**
- Schema "**LEFT JOIN + IS NULL**" = trovare ciò che **non** ha corrispondenze (clienti senza ordini, libri mai prestati...)

FULL OUTER JOIN e self join

MySQL non supporta `FULL OUTER JOIN`: si ottiene con l'unione di un LEFT e di un RIGHT JOIN.

```sql
SELECT I.Cognome, D.Descrizione
FROM Impiegati I LEFT JOIN Dipartimenti D ON I.Dipartimento = D.Codice
UNION
SELECT I.Cognome, D.Descrizione
FROM Impiegati I RIGHT JOIN Dipartimenti D ON I.Dipartimento = D.Codice;
```

Self join: una tabella congiunta con sé stessa (associazioni ricorsive). Le parti di una camicia nella distinta base `Oggetti(ID, Descrizione, Qta, ComponenteDi)`:

```sql
SELECT Prod.Descrizione AS Prodotto, Parti.Qta, Parti.Descrizione AS Componente
FROM Oggetti AS Parti
  JOIN Oggetti AS Prod ON Parti.ComponenteDi = Prod.ID
WHERE Prod.Descrizione = 'Camicia';
```

Funzioni di aggregazione

Funzione	Restituisce
`COUNT(*)`	numero di **righe**
`COUNT(colonna)`	numero di valori **non NULL**
`COUNT(DISTINCT colonna)`	numero di valori **diversi**
`SUM`, `AVG`	somma, media (ignorano i NULL)
`MIN`, `MAX`	minimo, massimo (anche su date e stringhe)

```
SELECT COUNT(*)                    AS NumImpiegati,
       COUNT(Dipartimento)         AS Assegnati,
       COUNT(DISTINCT Residenza)   AS Citta,
       ROUND(AVG(Stipendio), 2)    AS Media,
       MAX(Stipendio)              AS Massimo
FROM Impiegati;
```

Il risultato è **una sola riga**: le funzioni "riassumono" tutta la tabella.

GROUP BY: aggregare per gruppi

```sql
-- per ogni dipartimento: numero di impiegati e totale stipendi
SELECT D.Descrizione,
       COUNT(I.ID)        AS NumDipendenti,
       SUM(I.Stipendio)   AS TotStipendi
FROM Dipartimenti D
  LEFT JOIN Impiegati I ON D.Codice = I.Dipartimento
GROUP BY D.Codice, D.Descrizione
ORDER BY NumDipendenti DESC;
```

- Si forma **un gruppo** per ogni valore distinto delle colonne di `GROUP BY`; le funzioni si calcolano **per gruppo**
- Con `LEFT JOIN` e `COUNT(I.ID)` i dipartimenti vuoti compaiono con **0** (con `COUNT(*)` risulterebbe 1!)

Regola di MySQL (`ONLY_FULL_GROUP_BY`, attiva di default): ogni colonna nella `SELECT` che **non** è dentro una funzione di aggregazione deve comparire nel `GROUP BY` (errore `1055`).

HAVING: condizioni sui gruppi

```sql
-- dipartimenti con almeno 3 dipendenti e stipendio medio sopra 35.000 €
SELECT Dipartimento,
       COUNT(*)        AS NumDip,
       AVG(Stipendio)  AS Media
FROM Impiegati
WHERE Dipartimento IS NOT NULL         -- filtra le RIGHE prima di raggruppare
GROUP BY Dipartimento
HAVING COUNT(*) >= 3 AND AVG(Stipendio) > 35000;   -- filtra i GRUPPI
```

	`WHERE`	`HAVING`
Agisce su	singole righe	gruppi già formati
Quando	prima di `GROUP BY`	dopo `GROUP BY`
Funzioni di aggregazione	**non** ammesse	ammesse

Condizioni di ricerca: BETWEEN, IN, LIKE

```sql
SELECT * FROM Impiegati WHERE Stipendio BETWEEN 30000 AND 45000;  -- estremi inclusi
SELECT * FROM Impiegati WHERE Residenza IN ('Torino', 'Milano', 'Genova');
SELECT * FROM Impiegati WHERE Residenza NOT IN ('Roma');
SELECT * FROM Impiegati WHERE Cognome LIKE 'R%';                  -- inizia con R
```

Modello `LIKE`	Trova
`'Ros%'`	stringhe che **iniziano** con "Ros" (anche "Ros")
`'%ini'`	stringhe che **finiscono** con "ini"
`'%ar%'`	stringhe che **contengono** "ar"
`'_ossi'`	5 caratteri che finiscono con "ossi" (Rossi, Bossi…)
`'%\_%'`	stringhe che contengono il carattere `_` (con escape `\`)

`%` = zero o più caratteri · `_` = esattamente un carattere. Con la collation predefinita `LIKE` **non distingue** maiuscole e minuscole.

Subquery (interrogazioni annidate)

Una `SELECT` dentro un'altra: il risultato interno diventa un **valore** o un **insieme** per la query esterna.

```sql
-- impiegati con stipendio superiore alla media aziendale
SELECT Cognome, Nome, Stipendio
FROM Impiegati
WHERE Stipendio > (SELECT AVG(Stipendio) FROM Impiegati);

-- impiegati che partecipano al progetto 'Domo'
SELECT Cognome, Nome
FROM Impiegati
WHERE ID IN (SELECT ID FROM Partecipazioni WHERE CodProg = 'Domo');

-- impiegati che guadagnano più di TUTTI quelli del magazzino
SELECT Cognome, Stipendio
FROM Impiegati
WHERE Stipendio > ALL (SELECT Stipendio FROM Impiegati WHERE Dipartimento = 'Mag');
```

Una subquery confrontata con `=`, `>`, `<` deve restituire **un solo valore**.

IN, EXISTS, ANY, ALL

Predicato	Vero quando...
`x IN (subquery)`	x è **uno** dei valori restituiti
`x > ANY (subquery)`	x è maggiore di **almeno uno** (cioè del minimo)
`x > ALL (subquery)`	x è maggiore di **tutti** (cioè del massimo)
`EXISTS (subquery)`	la subquery restituisce **almeno una riga**

```sql
-- progetti a cui NON partecipa nessuno (subquery correlata)
SELECT P.CodProg, P.NomeProgetto
FROM Progetti P
WHERE NOT EXISTS (SELECT * FROM Partecipazioni Pa WHERE Pa.CodProg = P.CodProg);
```

Attenzione a `NOT IN`: se la subquery restituisce anche un solo **NULL**, il risultato è **vuoto**. Con `NOT EXISTS` (o LEFT JOIN + IS NULL) il problema non c'è.

Operazioni insiemistiche

Le due query devono avere lo **stesso numero di colonne**, con tipi compatibili.

```
-- città in cui c'è una sede OPPURE risiede un dipendente
SELECT Sede FROM Dipartimenti
UNION                       -- elimina i duplicati (UNION ALL li mantiene)
SELECT Residenza FROM Impiegati;

-- città che sono sia sede sia residenza       (MySQL 8.0.31 e successive)
SELECT Sede FROM Dipartimenti
INTERSECT
SELECT Residenza FROM Impiegati;

-- sedi in cui non risiede nessun dipendente   (MySQL 8.0.31 e successive)
SELECT Sede FROM Dipartimenti
EXCEPT
SELECT Residenza FROM Impiegati;
```

Nelle versioni precedenti, `INTERSECT` ed `EXCEPT` si scrivono con `IN` / `NOT IN` (o `EXISTS`).

Le viste (VIEW)

Una **vista** è una **tabella virtuale** definita da una `SELECT` salvata con un nome. Non contiene dati propri: ogni volta li ricava dalle tabelle di base.

```sql
CREATE VIEW PersonaleDipartimenti (Dipart, NumDipendenti, TotStipendi) AS
SELECT Dipartimento, COUNT(*), SUM(Stipendio)
FROM Impiegati
GROUP BY Dipartimento;

SELECT * FROM PersonaleDipartimenti WHERE NumDipendenti > 2;   -- si usa come una tabella

CREATE OR REPLACE VIEW ... ;      -- ridefinisce la vista
DROP VIEW PersonaleDipartimenti;
```

A cosa servono: **semplificare** query complesse, offrire a ogni utente una **visione parziale** dei dati (sottoschema), **proteggere** colonne riservate (insieme a `GRANT`).

6 · DCL: utenti e permessi

`CREATE USER` · `GRANT` · `REVOKE` · ruoli

Gli utenti in MySQL

Un account MySQL è formato da **nome utente + host** da cui si collega: `'utente'@'host'`.

```
CREATE USER 'segreteria'@'localhost' IDENTIFIED BY 'Segr#2026!';
CREATE USER 'direzione'@'%'           IDENTIFIED BY 'Dir#2026!';  -- % = da qualsiasi host

ALTER USER 'segreteria'@'localhost' IDENTIFIED BY 'NuovaPwd#1';   -- cambio password
DROP USER 'direzione'@'%';

SELECT user, host FROM mysql.user;      -- elenco degli account
```

- `'localhost'`: solo dalla stessa macchina del server (es. l'applicazione PHP)
- Il **principio del minimo privilegio**: l'applicazione web non deve mai collegarsi come `root`

GRANT e REVOKE

```sql
-- privilegi su tutte le tabelle del database
GRANT SELECT, INSERT, UPDATE, DELETE ON azienda.* TO 'segreteria'@'localhost';

-- su una sola tabella / su alcune colonne
GRANT SELECT ON azienda.Progetti TO 'direzione'@'%';
GRANT UPDATE (Residenza, Stipendio) ON azienda.Impiegati TO 'paghe'@'localhost';

-- su una vista (sottoschema)
GRANT SELECT ON azienda.PersonaleDipartimenti TO 'direzione'@'%';

-- con facoltà di concedere a sua volta il privilegio
GRANT SELECT ON azienda.* TO 'responsabile'@'localhost' WITH GRANT OPTION;

REVOKE DELETE ON azienda.* FROM 'segreteria'@'localhost';
SHOW GRANTS FOR 'segreteria'@'localhost';
```

Livelli: `*.*` (globale) → `db.*` (database) → `db.tabella` → `(colonne)`.

Privilegi più usati: `SELECT`, `INSERT`, `UPDATE`, `DELETE`, `CREATE`, `ALTER`, `DROP`, `EXECUTE`, `ALL PRIVILEGES`.

I ruoli (MySQL 8)

Un **ruolo** è un insieme di privilegi con un nome: si assegna il ruolo agli utenti invece di ripetere i `GRANT`.

```
CREATE ROLE 'lettore', 'operatore';

GRANT SELECT ON azienda.* TO 'lettore';
GRANT SELECT, INSERT, UPDATE ON azienda.* TO 'operatore';

GRANT 'lettore'   TO 'direzione'@'%';
GRANT 'operatore' TO 'segreteria'@'localhost';

-- i ruoli vanno ATTIVATI: così sono attivi a ogni accesso
SET DEFAULT ROLE ALL TO 'direzione'@'%', 'segreteria'@'localhost';
```

Nell'elaborato d'esame si può proporre: un utente **amministratore**, un utente **applicazione web** (solo DML sulle tabelle necessarie) e un utente **consultazione** (solo `SELECT`, magari su viste).

7 · Transazioni e concorrenza

ACID · `COMMIT` / `ROLLBACK` · livelli di isolamento · record fantasma

Che cos'è una transazione

> Una **transazione** è una sequenza di operazioni che il DBMS tratta come **un'unica unità**: o vanno a buon fine **tutte**, o **nessuna**.

Esempio classico, il bonifico: togliere 100 € dal conto A **e** aggiungerli al conto B. Se il sistema si blocca a metà, i soldi non devono sparire.

Proprietà	Significato
Atomicità	tutto o niente
Consistenza	si passa da uno stato valido a un altro stato valido (vincoli rispettati)
Isolamento	transazioni contemporanee non si disturbano
Durabilità	dopo il `COMMIT` le modifiche sopravvivono anche a un guasto

In MySQL le transazioni funzionano solo con tabelle **InnoDB**.

Transazioni in MySQL

MySQL lavora in **autocommit**: ogni istruzione è già una transazione confermata da sola. Per raggruppare più istruzioni:

```sql
START TRANSACTION;                          -- (oppure BEGIN)
UPDATE Conti SET Saldo = Saldo - 100 WHERE IdConto = 1;
UPDATE Conti SET Saldo = Saldo + 100 WHERE IdConto = 2;
COMMIT;                                     -- conferma definitivamente
-- ROLLBACK;                                -- oppure annulla tutto
```

```sql
START TRANSACTION;
INSERT INTO Progetti (CodProg, NomeProgetto, Durata) VALUES ('Smart', 'Smart city', 18);
SAVEPOINT progetto_creato;
INSERT INTO Partecipazioni VALUES (7, 'Smart', 'Direttore', 40);
ROLLBACK TO SAVEPOINT progetto_creato;      -- annulla solo ciò che segue il savepoint
COMMIT;                                     -- il progetto resta, la partecipazione no
```

I comandi **DDL** (`CREATE`, `ALTER`, `DROP`, `TRUNCATE`) causano un **COMMIT implicito**: non si possono annullare.

Le anomalie della concorrenza

Quando più utenti lavorano insieme, senza isolamento possono accadere:

Anomalia	Che cosa succede
Lettura sporca (*dirty read*)	T1 legge un dato modificato da T2 **non ancora confermato**; poi T2 fa `ROLLBACK` → T1 ha letto un valore mai esistito
Lettura non ripetibile	T1 legge una riga, T2 la **modifica** e conferma; T1 la rilegge e trova un **valore diverso**
Lettura fantasma (*phantom read*)	T1 legge un **insieme** di righe con una condizione; T2 **inserisce** (o cancella) una riga che soddisfa la condizione; T1 ripete la lettura e compare una riga **"fantasma"**
Aggiornamento perso	T1 e T2 leggono lo stesso valore e lo aggiornano: la scrittura di una **cancella** quella dell'altra

Un record fantasma: esempio

Un evento ha **100 posti**. Due utenti prenotano l'ultimo posto **nello stesso istante**.

Tempo	Sessione A	Sessione B
t1	`START TRANSACTION;`	`START TRANSACTION;`
t2	`SELECT COUNT(*) FROM Prenotazioni WHERE IdEvento = 7;` → **99**	
t3		`SELECT COUNT(*) …` → **99**
t4	`INSERT INTO Prenotazioni …;`	
t5		`INSERT INTO Prenotazioni …;`
t6	`COMMIT;`	`COMMIT;` → **101 prenotazioni!**

Per B, la riga inserita da A è un **fantasma**: non c'era quando ha contato, ma esiste quando conferma. Il controllo "ci sono posti?" si basava su un insieme di righe che **nel frattempo è cambiato**.

I livelli di isolamento

Livello	Lettura sporca	Non ripetibile	Fantasma
`READ UNCOMMITTED`	possibile	possibile	possibile
`READ COMMITTED`	evitata	possibile	possibile
`REPEATABLE READ` (**default MySQL**)	evitata	evitata	evitata per le letture semplici (vedi slide successiva)
`SERIALIZABLE`	evitata	evitata	evitata

```
SELECT @@transaction_isolation;                          -- livello attuale
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;   -- per tutta la sessione
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;             -- solo per la prossima transazione
```

Più isolamento = più sicurezza, ma anche più **attese** (lock) e meno prestazioni.

Come InnoDB gestisce i fantasmi

1. Letture semplici → snapshot (MVCC)

In `REPEATABLE READ` una `SELECT` normale legge una **fotografia** del database scattata alla prima lettura della transazione: le righe inserite da altri **non compaiono**. Utile per i report, ma **non basta** per decidere se inserire.

2. Letture con blocco → next-key lock

`SELECT … FOR UPDATE` (o `FOR SHARE`) legge i dati **più recenti** e blocca le righe trovate **e gli "spazi" tra di esse**: gli altri non possono inserire righe in quell'intervallo fino al `COMMIT`.

```
START TRANSACTION;
SELECT COUNT(*) INTO @occupati FROM Prenotazioni
WHERE IdEvento = 7 FOR UPDATE;          -- B, se fa lo stesso, ASPETTA qui
-- se @occupati < 100 allora:
INSERT INTO Prenotazioni (IdEvento, IdUtente) VALUES (7, 42);
COMMIT;                                  -- ora B riparte e conta 100
```

Serve un **indice** sulla colonna della condizione (qui `IdEvento`, già indicizzata dalla FK), altrimenti il blocco si estende a molte più righe.

8 · Stored routine e trigger

Codice salvato **nel** database ed eseguito **dal** server

Stored procedure e funzioni

> Una **stored routine** è un blocco di istruzioni SQL con un nome, salvato nel database ed eseguito dal server.
> **Procedura** (`PROCEDURE`): si esegue con `CALL`, può restituire risultati e parametri `OUT`.
> **Funzione** (`FUNCTION`): restituisce **un valore** con `RETURN` e si usa dentro le espressioni.

Vantaggi:

- **riuso**: la logica sta in un solo posto, utilizzabile da tutte le applicazioni
- **sicurezza**: l'utente può avere solo il privilegio `EXECUTE` sulla procedura, non l'accesso diretto alle tabelle
- **prestazioni**: meno traffico tra client e server
- **integrità**: regole complesse applicate sempre allo stesso modo

La prima procedura

```
DELIMITER //                    -- cambia il terminatore: il ; serve dentro il corpo

CREATE PROCEDURE ImpiegatiDi (IN p_dip CHAR(5))
BEGIN
  SELECT ID, Cognome, Nome, Stipendio
  FROM Impiegati
  WHERE Dipartimento = p_dip
  ORDER BY Cognome;
END //

DELIMITER ;                     -- ripristina il terminatore normale

CALL ImpiegatiDi('Prod');
```

- `DELIMITER` è un comando del **client** (riga di comando, Workbench): senza, il primo `;` chiuderebbe la `CREATE PROCEDURE`
- Parametri: `IN` (ingresso, default), `OUT` (uscita), `INOUT` (entrambi)
- `SHOW PROCEDURE STATUS WHERE Db = 'azienda';` · `DROP PROCEDURE ImpiegatiDi;`

Variabili, parametri OUT e controllo di flusso

```
DELIMITER //
CREATE PROCEDURE AumentaStipendi (IN  p_dip  CHAR(5),
                                  IN  p_perc DECIMAL(5,2),
                                  OUT p_righe INT)
BEGIN
  IF p_perc <= 0 OR p_perc > 20 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Percentuale non valida (1-20)';
  END IF;

  UPDATE Impiegati
  SET Stipendio = Stipendio * (1 + p_perc / 100)
  WHERE Dipartimento = p_dip;

  SET p_righe = ROW_COUNT();       -- righe modificate dall'ultima istruzione
END //
DELIMITER ;

CALL AumentaStipendi('Prod', 5, @n);
SELECT @n AS ImpiegatiAggiornati;
```

`SIGNAL SQLSTATE '45000'` genera un **errore personalizzato** e interrompe la procedura. Disponibili anche `DECLARE`, `CASE`, `WHILE`, `REPEAT`, `LOOP`.

Una funzione

```
DELIMITER //
CREATE FUNCTION StipendioMedio (p_dip CHAR(5))
RETURNS DECIMAL(9,2)
READS SQL DATA                    -- dichiara che legge ma non modifica i dati
BEGIN
  DECLARE v_media DECIMAL(9,2);
  SELECT AVG(Stipendio) INTO v_media
  FROM Impiegati WHERE Dipartimento = p_dip;
  RETURN IFNULL(v_media, 0);
END //
DELIMITER ;

-- si usa in un'espressione, come una funzione predefinita
SELECT Cognome, Stipendio,
       Stipendio - StipendioMedio(Dipartimento) AS ScartoDallaMedia
FROM Impiegati;
```

In MySQL 8 una funzione deve dichiarare `DETERMINISTIC`, `NO SQL` o `READS SQL DATA`, altrimenti la creazione dà errore `1418` (per via del registro binario attivo di default).

Procedura + transazione + gestione errori

Assegna un impiegato a un progetto solo se il suo impegno totale non supera il 100%.

```
DELIMITER //
CREATE PROCEDURE AssegnaProgetto (IN p_id SMALLINT UNSIGNED, IN p_prog CHAR(10),
                                  IN p_ruolo VARCHAR(30), IN p_perc TINYINT UNSIGNED)
BEGIN
  DECLARE v_tot INT;
  DECLARE EXIT HANDLER FOR SQLEXCEPTION      -- se qualcosa va storto...
  BEGIN
    ROLLBACK;                                -- ...annulla tutto
    RESIGNAL;                                -- ...e rilancia l'errore al chiamante
  END;

  START TRANSACTION;
  SELECT IFNULL(SUM(Percentuale), 0) INTO v_tot
  FROM Partecipazioni WHERE ID = p_id FOR UPDATE;   -- blocca: niente fantasmi
  IF v_tot + p_perc > 100 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Impegno totale oltre il 100%';
  END IF;
  INSERT INTO Partecipazioni VALUES (p_id, p_prog, p_ruolo, p_perc);
  COMMIT;
END //
DELIMITER ;
```

I trigger

> Un **trigger** è una routine eseguita **automaticamente** dal DBMS quando su una tabella avviene un **evento** (`INSERT`, `UPDATE`, `DELETE`), **prima** (`BEFORE`) o **dopo** (`AFTER`) l'operazione, **per ogni riga** coinvolta.

Evento	`OLD.colonna` (valore prima)	`NEW.colonna` (valore dopo)
`INSERT`	—	sì (modificabile nei `BEFORE`)
`UPDATE`	sì (sola lettura)	sì (modificabile nei `BEFORE`)
`DELETE`	sì (sola lettura)	—

Usi tipici:

- **`BEFORE`**: validare o correggere i dati prima che vengano scritti
- **`AFTER`**: registrare un **log/storico**, aggiornare totali in altre tabelle

Trigger BEFORE: validazione

Un vincolo `CHECK` vede solo la riga corrente; un trigger può consultare **altre righe**.

```
DELIMITER //
CREATE TRIGGER trg_part_check
BEFORE INSERT ON Partecipazioni
FOR EACH ROW
BEGIN
  DECLARE v_tot INT;
  SELECT IFNULL(SUM(Percentuale), 0) INTO v_tot
  FROM Partecipazioni WHERE ID = NEW.ID;

  IF v_tot + NEW.Percentuale > 100 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Impegno totale oltre il 100%';
  END IF;

  SET NEW.Ruolo = UPPER(TRIM(NEW.Ruolo));     -- normalizza il dato in ingresso
END //
DELIMITER ;
```

Il controllo ora vale per **qualsiasi** `INSERT`: da procedure, da PHP, da Workbench.

Trigger AFTER: lo storico degli stipendi

```
CREATE TABLE StoricoStipendi (
  IdMod      INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  ID         SMALLINT UNSIGNED NOT NULL,
  Vecchio    DECIMAL(9,2),
  Nuovo      DECIMAL(9,2),
  Utente     VARCHAR(100),
  DataOra    DATETIME DEFAULT CURRENT_TIMESTAMP
);

DELIMITER //
CREATE TRIGGER trg_storico_stipendi
AFTER UPDATE ON Impiegati
FOR EACH ROW
BEGIN
  IF NOT (OLD.Stipendio <=> NEW.Stipendio) THEN       -- <=> confronto "NULL-safe"
    INSERT INTO StoricoStipendi (ID, Vecchio, Nuovo, Utente)
    VALUES (OLD.ID, OLD.Stipendio, NEW.Stipendio, CURRENT_USER());
  END IF;
END //
DELIMITER ;
```

Ora `CALL AumentaStipendi('Prod', 5, @n);` lascia **una riga di storico per ogni impiegato**.

Trigger: limiti e buone pratiche

- In MySQL i trigger sono solo **a livello di riga** (`FOR EACH ROW`)
- Un trigger **non può modificare** la tabella su cui è definito con `INSERT`/`UPDATE`/`DELETE` (errore `1442`): per cambiare la riga corrente si usa `SET NEW.col` in un `BEFORE`
- Dentro un trigger **non** si usano `START TRANSACTION`, `COMMIT`, `ROLLBACK`: il trigger fa parte della transazione dell'istruzione che lo ha attivato. Se fallisce, **fallisce anche l'istruzione**
- `TRUNCATE` **non** attiva i trigger `DELETE`
- Più trigger sullo stesso evento: ordine con `FOLLOWS` / `PRECEDES`

```
SHOW TRIGGERS FROM azienda;
DROP TRIGGER IF EXISTS trg_part_check;
```

Logica "nascosta": troppi trigger rendono il database difficile da capire. Usarli per **integrità** e **audit**, documentandoli.

9 · Un caso completo

Le attività sportive studentesche: dallo schema alle query in MySQL

Il problema

Gli **studenti** di vari **istituti** partecipano a **manifestazioni** sportive. Ogni istituto indica dei **professori** come allenatori: un professore segue **una sola** manifestazione, una manifestazione può avere professori di scuole diverse.

Schema logico:

- Istituti (CodiceIstituto, Denominazione, Indirizzo, Telefono)
- Studenti (CodiceStud, CognStud, NomeStud, DataNascita, Classe, *CodiceIstituto*)
- Manifestazioni (CodiceManif, Descrizione, Luogo, DataInizio)
- Professori (CodiceProf, CognProf, NomeProf, *CodiceIstituto*, *CodiceManif*)
- Iscrizioni (ID, *CodiceStud*, *CodiceManif*, DataIscrizione)

Vincolo: uno studente si iscrive **una sola volta** a ogni manifestazione → `UNIQUE (CodiceStud, CodiceManif)`.

Lo script DDL (estratto)

```
CREATE TABLE Istituti (
  CodiceIstituto  CHAR(4)      PRIMARY KEY,
  Denominazione   VARCHAR(50)  NOT NULL UNIQUE,
  Indirizzo       VARCHAR(60),
  Telefono        VARCHAR(15)
);
CREATE TABLE Studenti (
  CodiceStud      INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  CognStud        VARCHAR(30) NOT NULL,
  NomeStud        VARCHAR(30) NOT NULL,
  DataNascita     DATE,
  Classe          TINYINT UNSIGNED CHECK (Classe BETWEEN 1 AND 5),
  CodiceIstituto  CHAR(4) NOT NULL,
  FOREIGN KEY (CodiceIstituto) REFERENCES Istituti (CodiceIstituto)
);
CREATE TABLE Iscrizioni (
  ID              INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  CodiceStud      INT UNSIGNED NOT NULL,
  CodiceManif     CHAR(3)      NOT NULL,
  DataIscrizione  DATE DEFAULT (CURRENT_DATE),
  UNIQUE (CodiceStud, CodiceManif),
  FOREIGN KEY (CodiceStud)  REFERENCES Studenti (CodiceStud) ON DELETE CASCADE,
  FOREIGN KEY (CodiceManif) REFERENCES Manifestazioni (CodiceManif)
);
```

Le interrogazioni (1)

```sql
SET @manif = 'Campionato di pallavolo';

-- numero di studenti iscritti a una manifestazione
SELECT COUNT(*) AS NumeroStudenti
FROM Manifestazioni M JOIN Iscrizioni I ON M.CodiceManif = I.CodiceManif
WHERE M.Descrizione = @manif;

-- elenco alfabetico degli allenatori di quella manifestazione
SELECT P.CognProf, P.NomeProf
FROM Manifestazioni M JOIN Professori P ON M.CodiceManif = P.CodiceManif
WHERE M.Descrizione = @manif
ORDER BY P.CognProf, P.NomeProf;

-- scuole con studenti iscritti a quella manifestazione
SELECT DISTINCT T.Denominazione, T.Indirizzo, T.Telefono
FROM Manifestazioni M
  JOIN Iscrizioni I ON M.CodiceManif = I.CodiceManif
  JOIN Studenti S   ON I.CodiceStud  = S.CodiceStud
  JOIN Istituti T   ON S.CodiceIstituto = T.CodiceIstituto
WHERE M.Descrizione = @manif;
```

Le interrogazioni (2)

```sql
-- per ogni scuola, numero di studenti che partecipano ad almeno una manifestazione
-- (COUNT DISTINCT sulla colonna di Iscrizioni: le scuole senza partecipanti mostrano 0)
SELECT T.Denominazione, COUNT(DISTINCT I.CodiceStud) AS Partecipanti
FROM Istituti T
  LEFT JOIN Studenti S   ON T.CodiceIstituto = S.CodiceIstituto
  LEFT JOIN Iscrizioni I ON S.CodiceStud = I.CodiceStud
GROUP BY T.CodiceIstituto, T.Denominazione;

-- manifestazioni con più di 20 iscritti, dalla più partecipata
SELECT M.Descrizione, COUNT(*) AS Iscritti
FROM Manifestazioni M JOIN Iscrizioni I ON M.CodiceManif = I.CodiceManif
GROUP BY M.CodiceManif, M.Descrizione
HAVING COUNT(*) > 20
ORDER BY Iscritti DESC;

-- manifestazioni ancora SENZA iscritti
SELECT M.Descrizione
FROM Manifestazioni M LEFT JOIN Iscrizioni I ON M.CodiceManif = I.CodiceManif
WHERE I.ID IS NULL;
```

Errori frequenti in MySQL

Errore	Correzione
`REFERENCES` accanto alla colonna (in 8.x è ignorato)	clausola `FOREIGN KEY (…) REFERENCES …`
Tipi diversi tra FK e PK (`INT` vs `INT UNSIGNED`)	tipi **identici**
Date scritte `'10/11/2026'`	formato `'2026-11-10'`
`WHERE campo = NULL`	`WHERE campo IS NULL`
`'Rossi' || ' ' || 'Mario'`	`CONCAT('Rossi', ' ', 'Mario')`
Colonna non aggregata assente dal `GROUP BY` (errore 1055)	aggiungerla al `GROUP BY`
Condizione su `COUNT(*)` scritta in `WHERE`	spostarla in `HAVING`
`COUNT(*)` con `LEFT JOIN` per contare figli	`COUNT(colonna_del_figlio)`
Prezzi in `FLOAT`	`DECIMAL(p,2)`
`UPDATE` / `DELETE` senza `WHERE`	controllare **sempre** la condizione

Esercizi

Database **biblioteca scolastica**:
`Libri(CodLibro, Titolo, Autore, Anno, Genere)` · `Utenti(Tessera, Cognome, Nome, Classe)` · `Prestiti(IdPrestito, CodLibro, Tessera, DataPrestito, DataRestituzione)` (`DataRestituzione` NULL = libro non ancora reso)

1. Scrivi il **DDL MySQL** delle tre tabelle con tipi, chiavi, `CHECK (Anno <= 2100)` e chiavi esterne
2. Elenca titoli e autori dei libri **attualmente in prestito**
3. Per ogni genere, il **numero di prestiti** effettuati, in ordine decrescente
4. Gli utenti che **non hanno mai** preso un libro in prestito
5. Crea un utente `'bibliotecario'@'localhost'` che può leggere tutto e inserire/modificare solo `Prestiti`
6. Scrivi un **trigger** che impedisca di prestare un libro **già in prestito**
7. Scrivi una **procedura** `RestituisciLibro(p_cod)` che imposti la data di restituzione a oggi

Soluzioni (2–6)

```sql
-- 2
SELECT L.Titolo, L.Autore FROM Libri L JOIN Prestiti P ON L.CodLibro = P.CodLibro
WHERE P.DataRestituzione IS NULL;
-- 3
SELECT L.Genere, COUNT(*) AS NumPrestiti FROM Libri L JOIN Prestiti P ON L.CodLibro = P.CodLibro
GROUP BY L.Genere ORDER BY NumPrestiti DESC;
-- 4
SELECT U.Tessera, U.Cognome, U.Nome FROM Utenti U LEFT JOIN Prestiti P ON U.Tessera = P.Tessera
WHERE P.IdPrestito IS NULL;
-- 5
CREATE USER 'bibliotecario'@'localhost' IDENTIFIED BY 'Bibl#2026!';
GRANT SELECT ON biblioteca.* TO 'bibliotecario'@'localhost';
GRANT INSERT, UPDATE ON biblioteca.Prestiti TO 'bibliotecario'@'localhost';
-- 6
DELIMITER //
CREATE TRIGGER trg_no_doppio_prestito BEFORE INSERT ON Prestiti FOR EACH ROW
BEGIN
  IF EXISTS (SELECT * FROM Prestiti
             WHERE CodLibro = NEW.CodLibro AND DataRestituzione IS NULL) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Libro già in prestito';
  END IF;
END //
DELIMITER ;
```

Soluzioni (1 e 7)

```sql
-- 1 (estratto: la tabella Prestiti)
CREATE TABLE Prestiti (
  IdPrestito        INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  CodLibro          INT UNSIGNED NOT NULL,
  Tessera           INT UNSIGNED NOT NULL,
  DataPrestito      DATE NOT NULL DEFAULT (CURRENT_DATE),
  DataRestituzione  DATE,
  CHECK (DataRestituzione IS NULL OR DataRestituzione >= DataPrestito),
  FOREIGN KEY (CodLibro) REFERENCES Libri (CodLibro),
  FOREIGN KEY (Tessera)  REFERENCES Utenti (Tessera)
);

-- 7
DELIMITER //
CREATE PROCEDURE RestituisciLibro (IN p_cod INT UNSIGNED)
BEGIN
  UPDATE Prestiti SET DataRestituzione = CURDATE()
  WHERE CodLibro = p_cod AND DataRestituzione IS NULL;
  IF ROW_COUNT() = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Nessun prestito aperto per questo libro';
  END IF;
END //
DELIMITER ;
```

Riepilogo

DDL `CREATE` · `ALTER` · `DROP`: tipi corretti, vincoli e `FOREIGN KEY` sempre in forma separata

DML `INSERT` · `UPDATE` · `DELETE`: sempre con `WHERE`

SELECT: `JOIN`, `GROUP BY` / `HAVING`, subquery, viste

DCL `CREATE USER` · `GRANT` · `REVOKE` · ruoli: minimo privilegio

TCL `START TRANSACTION` · `COMMIT` · `ROLLBACK`: ACID, `FOR UPDATE` contro i fantasmi

Stored routine e trigger: la logica e i controlli vivono nel database